# Discovery Executive Summary

**Project:** discovery-pingCRM-7Aug · **Generated:** 07/08/2026, 17:16:12

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

## Portfolio Overview

| # | Analysis | Overall Rating |
|---|---|---|
| 1 | Architecture & Design Analysis | — |
| 2 | Code Quality & Complexity Analysis | — |

---

## 1. Architecture & Design Analysis

EXECUTIVE SUMMARY

This is a Laravel 11 + Inertia/React application (PingCRM base) that has been overgrown with a large, deliberately-legacy "IVR" subsystem. Architectural health is **High Risk**: 83 single-action IVR controllers each carry ~759 lines of inline business logic, raw string-concatenated `DB::select` queries, `extract($request->all())`, hard-coded tenant IDs, and `new SomethingGodService()` instantiation — so controllers, persistence, and domain rules are fused into one untestable layer. A Repository tier (12 classes) and a Service tier (12 `*GodService` classes) both exist but are architecturally hollow: the repositories are referenced by zero call-sites (dead abstraction) and the services are god-classes with 45 unrelated public methods, mutable `static` state, and embedded secrets. There is no Dependency Injection wiring at all (`AppServiceProvider` binds nothing; `Model::unguard()` is global), so nothing can be substituted or mocked. On the frontend, 916 components repeat the same pathology in React terms — 374 components issue inline `fetch()` calls to hard-coded URLs, 147 legacy class components mix paradigms, and 8 near-identical 1,101-line "formatter" modules duplicate the same logic. The dominant risks are **change amplification** (one schema or rule change touches dozens of controllers, services, repos and components at once) and **hidden coupling** (reporting reads other domains' operational tables directly), both of which will make any modernization or team scaling extremely costly until bounded contexts, a real service layer, and DI are introduced.

## 1.1 Benchmark Ratings Summary

| # | Hotspot | Primary KPI | Good | Moderate | High Risk | Measured |
|---|---|---|---|---|---|---|
| H1 | Fat Controllers | Avg LOC per controller | <150 | 150–300 | >300 | **~683 LOC avg** (81 of 91 > 300 LOC; IVR = 759 LOC each) |
| H2 | Missing Service Layer | Controllers accessing repos/models directly | <10 | 10–20 | >20 | **83** controllers with inline DB/model access + rules |
| H3 | Missing Repository Pattern | Direct DB access points outside repositories | <10 | 10–20 | >20 | **95** files use `DB::` directly; 12 repositories have **0** references |
| H4 | Circular Dependencies | Dependency cycles | 0 | 1–3 | >3 | **0** cycles observed |
| H5 | Shared Utility Abuse | Utility files w/ business logic | 0 | 1–5 | >5 | **13** (5 backend helpers + 8 frontend dup formatters) |
| H6 | Direct SQL in Controllers | ORM/repository compliance % | >90% | 60–90% | <60% | **~9%** compliant (83 of 91 controllers embed raw SQL) |
| H7 | God Classes | Classes >1000 LOC | 0 | 1–3 | >3 | 0 backend > 1000 LOC, **but** 12 `*GodService` @ **45** methods + static state |
| H8 | Domain Boundary Violations | Cross-domain access points | 0 | 1–5 | >5 | **4** (Reports, IvrHub, LoadsIvrModuleData, IvrAccountContext) |
| H9 | Shared Database Coupling | Tables shared across domains | <10% | 10–30% | >30% | **~21%** (3 of 14 tables shared, no ownership) |
| H10 *(additional)* | No Dependency Injection / Service Locator | Services resolved via `new` | 0 | 1–10 | >10 | **80** `new *GodService()` ; 0 container bindings |
| F1 | Business Logic in Components | Avg LOC per component | <150 | 150–300 | >300 | avg **139 LOC**; 134 of 522 pages (26%) > 300 LOC |
| F2 | Missing Frontend Service/Data Layer | Components w/ inline API calls | <10 | 10–20 | >20 | **374** components with inline `fetch()` /hard-coded URLs |

| | | | | | | |
|---|---|---|---|---|---|---|
| F3 | God / Oversized Components | Components >400 LOC | 0 | 1–3 | >3 | **1** ( `Hub/Index.tsx` = 479 LOC); 134 more in 300–400 band |
| F4 | Prop Drilling / Global State Abuse | Max prop-drilling depth | ≤2 | 3–4 | >4 | **≤2** — components self-fetch into local state |
| F5 | Legacy / Inconsistent Component Patterns | Legacy-pattern components | 0 | 1–10 | >10 | **147** class components mixed with function components |

*No additional hotspots beyond H10 were observed.*

## 1.4 Actions Required

| Hotspot | Action | Rating | Priority |
|---|---|---|---|
| H1 Fat Controllers | Reduce the 83 IVR controllers to validate-and-delegate; extract logic into Application Services | High Risk | Critical |
| H2 Missing Service Layer | Introduce per-capability Application Services; move query/workflow assembly out of controllers | High Risk | Critical |
| H6 Direct SQL in Controllers | Move all `DB::` queries out of controllers into bound-parameter repositories; add CI grep gate | High Risk | Critical |
| H10 No Dependency Injection | Bind interfaces in `AppServiceProvider`, inject services, remove `new *GodService()` and `Model::unguard()` | High Risk | Critical |
| H3 Missing Repository Pattern | Give repositories interfaces + bound queries; route all persistence through them | High Risk | High |
| H5 Shared Utility Abuse | Replace 5 `LegacyIvr*` helpers with domain services; collapse 8 duplicate formatter modules into one | High Risk | High |
| H7 God Classes | Split each `*GodService` into domain service + repository + application service; remove static state/secret | High Risk | High |
| F2 Missing Frontend Service Layer | Introduce a typed API client + hooks; replace 374 inline `fetch()` /hard-coded URLs | High Risk | High |
| F5 Legacy Component Patterns | Codemod 147 class widgets to hooks; add error boundaries and a ban-class lint rule | High Risk | High |
| H8 Domain Boundary Violations | Introduce published read models / ACL; forbid cross-context table reads in Reports & Hub | Moderate | Medium |

| | | | |
|---|---|---|---|
| H9 Shared Database Coupling | Assign per-table ownership; serve reporting via an owned read model instead of live joins | Moderate | Medium |
| F1 Business Logic in Components | Move validation/transform into shared hooks/schema; split 300–400-LOC page templates | Moderate | Medium |
| F3 God / Oversized Components | Decompose `Hub/Index.tsx` (479 LOC) and the shared 392-LOC template into widgets + hooks | Moderate | Medium |

## 1.5 Expected Outcomes

- **Testable, reusable business logic** — with Application/Domain Services and constructor DI in place, workflows can be unit-tested with mocked repositories and reused from HTTP, CLI and queued jobs instead of being copy-pasted across 83 controllers.
- **Schema and persistence freedom** — funnelling every `DB::` call through parameter-bound repositories removes the injection holes and lets the team rename/repartition `ivr_*` tables without editing dozens of hand-written query strings.
- **Independent, extractable domains** — bounded contexts with published interfaces and an anti-corruption layer let Queue, Call Routing, Analytics and Reporting evolve (and eventually extract) without silently breaking each other through shared tables.
- **A consistent, maintainable frontend** — a single typed API client plus hooks and function components collapses 374 inline fetches and 147 class widgets into one convention, so endpoint/auth changes are one-file edits and screens stop leaking timers and blanking on errors.
- **Sharply lower change amplification** — de-duplicating the 5 helpers and 8 formatter modules and enforcing CI gates (no `DB::` in controllers, no `React.Component`, duplication check) means a single rule change stops rippling through dozens of near-identical files.","ttft_ms":7882,"ttft_stream_ms":7697,"time_to_request_ms":6140,"type":"result","duration_ms":491456,"uuid":"07a4177b-031e-4e68-a1fd-2230b3ef721b"}

---

## 2. Code Quality & Complexity Analysis

EXECUTIVE SUMMARY

This codebase is a Ping CRM base (Laravel + Inertia/React) onto which a very large "Legacy IVR" surface has been grafted, and that surface is dominated by copy-paste rather than genuine algorithmic complexity. Coverage spans **both layers**: 141 backend PHP files and 1,051 frontend TypeScript/TSX files (1,192 total). The most severe findings are structural, not branch-depth: the frontend contains 8 files over 1,100 LOC and 133 near-identical `LegacyPass2_*.tsx` pages, while the backend has 82 fat IVR controllers at exactly 759 LOC each, 12 `*GodService` classes, and 12 near-identical repositories — pushing estimated overall duplication well above 60%. By contrast, per-method cyclomatic complexity is genuinely low (max ≈ 8) and no single function exceeds ~22 LOC, so those hotspots rate Good. Git history was available (124 commits, 2020–2026); churn and defect-fix frequency on the maintained core are low, but note that the entire high-risk IVR surface arrived in a single bulk commit — so its near-zero churn is deceptive, not reassuring. Additional backend anti-patterns were confirmed: shared mutable `public static` state in all 12 God services and ~4,940 uses of `extract()` for dynamic variable creation.

## 2.1 Benchmark Ratings Summary

| # | Hotspot | Primary KPI | Good | Moderate | High Risk | Measured |
|---|---|---|---|---|---|---|
| H1 | High Cyclomatic Complexity | Max complexity per method | <10 | 10–20 | >20 | ~8 ( `IvrHubController::l` |
| H2 | Large Classes | Largest class/file LOC | <300 | 300–1000 | >1000 | 1,101 LOC (frontend); 759 (backend) |
| H3 | Large Functions | Largest function LOC | <50 | 50–200 | >200 | ~22 LOC ( `handleStore` |
| H4 | Business Logic Duplication | Duplicated business logic % | <5% | 5–10% | >10% | ~60% (handlers + workflo per module) |
| H5 | Duplicate Code (general) | Overall duplicate code % | <5% | 5–10% | >10% | ~65% (est., manual — no jscpd/phpcpd configured) |
| H6 | High Churn Areas | Monthly changes (top files) | <5 | 5–10 | >10 | ~0.35/mo (top file, 27 char mo) |
| H7 | Defect-Prone Files | Fix commits (hottest file) | 1–3 | 4–5 | >5 | 3 ( `Contacts/Index` ) |
| H8 | Ownership Issues | Top-author ownership % | >80% | 60–80% | <60% | 43–71% (maintained core |
| H9 | Global Mutable State *(additional)* | Shared mutable static/global holders (target 0) | 0 | 1–3 | >3 | 12 ( `public static $sharedRuntimeCache` ) |
| H10 | Dynamic Variable Creation *(additional)* | `extract()` /dynamic-var uses (target 0) | 0 | 1–50 | >50 | ~4,940 `extract($paylo` |

### Hotspot Score breakdown

| Component | Weight | Sub-score (0–100) | Weighted |
|---|---|---|---|
| Cyclomatic Complexity | 25% | 18 | 4.50 |
| Code Churn | 25% | 15 | 3.75 |

| | | | |
|---|---|---|---|
| Defect Density | 20% | 30 | 6.00 |
| Class/Function Size | 15% | 80 | 12.00 |
| Business Logic Duplication | 10% | 92 | 9.20 |
| Developer Ownership Risk | 5% | 55 | 2.75 |
| **Hotspot Score** | **100%** | | **38 / 100** |

## 2.5 Actions Required

| Hotspot | Action | Rating | Priority |
|---|---|---|---|
| H4 Business Logic Duplication | Consolidate per-module workflows into one `IvrModuleService` + a shared `useIvrRecordForm` hook | High Risk | Critical |
| H5 Duplicate Code (general) | Replace 133 `LegacyPass2` pages / 8 formatter clones / 12 repos with data-driven components; add `jscpd` / `phpcpd` CI gate | High Risk | Critical |
| H2 Large Classes | Split 82 fat IVR controllers into single-action controllers + services; collapse >1,000-LOC formatter modules | High Risk | High |
| H9 Global Mutable State | Replace `public static $sharedRuntimeCache` with request-scoped/per-tenant cache in all 12 God services | High Risk | High |
| H10 Dynamic Variable Creation | Replace ~4,940 `extract($payload)` calls with explicit access/DTOs; ban `extract()` via PHPStan | High Risk | High |
| H8 Ownership Issues | Add CODEOWNERS for CRM core and assign an owner/deprecation decision for `Legacy/**` and `Pages/Ivr/**` | Moderate | Medium |

## 2.6 Expected Outcomes

- **Lower maintenance cost:** consolidating duplicated workflows and files removes an estimated ~60% of redundant code, so each business-rule change is made once instead of across 12–230 sites.
- **Fewer regressions:** a single shared service + form hook eliminates the "fixed here but not there" defect class that dominates the IVR surface today.
- **Faster, safer builds:** deleting 8 oversized formatter modules and ~360 clone components shrinks the frontend bundle and CI time; a duplication gate prevents new clones.
- **Correctness & isolation:** removing shared static state and `extract()` closes cross-tenant leakage and restores static-analysis (Larastan) coverage.
- **Clearer ownership:** CODEOWNERS on the maintained core and an explicit decision on the Legacy/IVR surface reduce coordination overhead and stop dead code from accreting.

---

**Report saved to** `docs/discovery/02-code-quality-complexity.md` (both frontend and backend layers covered). The orchestration UI will convert it to the matching PDF.","ttft_ms":4694,"ttft_stream_ms":3705,"time_to_request_ms":125,"type":"result","duration_ms":452890,"uuid":"7c0f0161-e69f-4426-b88a-b19daa69a597"}